

**ZERO TRUST DIGITAL
PROVENANCE PLATFORM**

Project Progress and Deliverables Report

*Requirements traceability, implementation status, production readiness, and completion
roadmap*

Project code	FYP-SP-08
Programme	BS Computer Science - Final Year Project
Institution	FAST National University of Computer and Emerging Sciences, Karachi Campus
Supervisor	Moheez Ur Raheem
Project team	Muhammad Shaheer Zafar (22K-5138); Syed Hasham Uddin (22K-6013); Sohaib Sherwani (21K-4790)
Status date	20 September 2026
Report version	1.0

Prepared from SRS v2.0, SDS v1.0, FYP-I Report, repository inspection, and executed verification checks

Document Control

Item	Value
Purpose	Provide a formal, evidence-based progress snapshot and identify the work required for FYP completion and production readiness.
Assessment baseline	SRS v2.0 (10 May 2026), SDS v1.0, FYP-I Progress Report, and the implementation available on 20 September 2026.
Repositories reviewed	ZTDPP frontend; ZTDPP-Backend NestJS API; AI-Service FastAPI inference service; ForensicFusion training artefacts.
Status method	Complete = implemented and evidenced; Partial = implemented with gaps or superseded design; Not implemented = required deliverable absent.
Important limitation	Completion percentages are management estimates based on traceability evidence, not supervisor sign-off or an independent production certification.

Table of Contents

Document Control..... 2

Table of Contents 2

1. Executive Summary..... 4

 Immediate management decisions..... 4

2. Baseline and Scope Evolution 5

 2.1 Original approved baseline 5

 2.2 Current implemented architecture..... 5

 2.3 Major controlled deviations..... 5

3. Deliverable Progress Register 7

 Deliverable summary 8

4. Functional Requirement Traceability 9

5. Implemented System and Data Flows 10

 5.1 Identity and access..... 10

 5.2 Provenance registration and lineage 10

 5.3 Verification and metadata return 10

 5.4 Storage map 11

6. Quality, Security, and Production Evidence..... 12

6.1 Verification evidence executed.....	12
6.2 AI model status	12
6.3 Security posture	13
7. Gap and Risk Register.....	13
8. Completion Roadmap	15
Phase A - Production blockers	15
Phase B - AI quality and explainability	15
Phase C - Deliverables and governance	15
9. Proposed Acceptance Gates	15
10. Final Assessment.....	16
Appendix A - Source and Evidence Register	16
Appendix B - Status Definitions.....	17

1. Executive Summary

ZTDPP has progressed from the original proof-of-concept design into a substantially broader platform. The current system includes account security, developer API keys, signed provenance records, versioned asset lineage, a platform-operated hash-chained ledger, perceptual matching, persisted verification reports, an administrative console, and a separately deployable AI inference service.

The implementation is stronger than the original baseline in several operational areas, but it no longer follows the SRS/SDS architecture literally. The original Python FastAPI and SQLite orchestrator has become a NestJS and MongoDB platform; ECDSA signatures embedded in EXIF have become server-side Ed25519-signed manifests and ledger receipts; EfficientNet-B4/ONNX has become a PyTorch ConvNeXt/residual model; and React/Vite has become Next.js.

Overall assessment: 12 of 21 scheduled deliverables are complete, 7 are partial or superseded, and 2 are not implemented. Using a simple 1.0/0.5/0 weighting, literal baseline completion is approximately 74%. Core platform capability is materially further advanced than that number suggests, but public production deployment remains conditional on security closure, AI robustness, formal performance evidence, and scope re-baselining.

Dimension	Assessment	Evidence-based conclusion
Literal SRS/Gantt completion	74%	12 complete, 7 partial/superseded, 2 not implemented.
Core provenance platform	Advanced	Registration, edits, revocation, signatures, ledger, verification, history, API keys, and dashboards are operational in code.
AI integration	Integrated; model risk remains	FastAPI service and checkpoint load correctly, but an edited Gemini sample produced a severe false negative.
Production readiness	Conditional	Containerization, CI, security middleware, health checks, and tests exist; critical deployment and validation gates remain.
Documentation alignment	Needs re-baseline	SRS/SDS describe a materially different runtime architecture and cryptographic protocol.

Immediate management decisions

1. Approve an SRS/SDS v3.0 change baseline that records the current NestJS, MongoDB, Ed25519, ledger, Next.js, and PyTorch architecture.
2. Treat GradCAM and PDF forensic reporting as explicit pending deliverables or formally remove them from the approved scope.
3. Do not promote the current AI checkpoint as a reliable cross-generator detector until it passes generator-held-out and post-processing regression tests.

4. Close the unauthenticated role-creation route and complete the production secret, signing-key, database, and rate-limiter deployment controls before public exposure.
5. Execute and preserve results for the S1-S7 adversarial suite, end-to-end tests, and latency/load benchmarks before final acceptance.

2. Baseline and Scope Evolution

2.1 Original approved baseline

The SRS v2.0 defines eleven use cases and a proof-of-concept built around a Python signing utility, FastAPI orchestration, SQLite trusted-device registry, ECDSA P-256 signatures embedded in EXIF, EfficientNet-B4 inference exported to ONNX, GradCAM explanations, a React SPA, API keys, and verification history.

2.2 Current implemented architecture

- Frontend: Next.js 15 dashboard with user and administrator workspaces, 27 page routes, Axios integration, responsive trust reports, lineage views, key management, and operational settings.
- Core API: NestJS 11 service with MongoDB/Mongoose, JWT session cookies, OTP flows, role-based administration, request tracing, throttling, image validation, and Swagger documentation.
- Provenance: content hashing, perceptual hashing, C2PA-aligned assertions, signed manifests, asset version lineage, ownership controls, revocation events, and exact/near matching.
- Ledger: ordered entries, previous-entry links, Merkle roots and proofs, proof-of-work blocks, Ed25519 signatures, public verification endpoints, and scheduled/manual sealing.
- AI: separate FastAPI service loading a trusted PyTorch ForensicFusion checkpoint with ConvNeXt Tiny plus a fixed high-pass residual stream, calibrated threshold, API-key protection, health probes, and upload safety limits.
- Deployment: Docker Compose for development and production, Caddy TLS reverse proxy, non-root/capability-restricted AI container, resource limits, environment templates, and GitHub Actions workflows.

2.3 Major controlled deviations

Area	SRS/SDS baseline	Current implementation	Assessment
Core backend	FastAPI orchestrator	NestJS core API; FastAPI retained only for AI	Scope changed; operational separation improved
Persistence	SQLite registry/history	MongoDB replica-set transactions and domain collections	Scope changed; more scalable, requires formal update
Cryptography	ECDSA P-256 in EXIF	Ed25519 platform signatures and signed ledger blocks	Protocol changed; stronger chain model, not literal C2PA

Area	SRS/SDS baseline	Current implementation	Assessment
Provenance storage	Manifest embedded into image	Manifest stored and returned by platform; container presence detected	Partial against UC-01; full embedding absent
AI architecture	EfficientNet-B4 exported to ONNX	ConvNeXt Tiny plus residual stream served in PyTorch	Superseded implementation; ONNX target unmet
Frontend	React 18 with Vite	Next.js 15 App Router	Superseded with broader capability
Verdicts	Verified / Unverified / Blocked	Trusted / Likely authentic / Suspicious / Untrusted	Policy model expanded; documents must be updated
Explainability	GradCAM heatmap	No GradCAM endpoint or heatmap response	Open deliverable

3. Deliverable Progress Register

The following register maps Appendix D of SRS v2.0 to the current workspace. 'Complete' means the capability exists and has direct code or test evidence. 'Partial' includes scope substitutions that meet the business intent but do not meet the documented design literally.

ID	Deliverable	Status	Current evidence
T1.1	Dataset acquisition and preprocessing	Complete	ForensicFusion metadata ingestion, content hashing, balanced sampling, split validation, and transforms are implemented.
T2.1	UI/UX wireframes and JSON schema	Complete	Production-style Next.js interface and stable API response conventions supersede the initial wireframes.
T3.1	Backend architecture and registry schema	Complete	NestJS domain modules and MongoDB entities replace the original SQLite schema.
T2.2	Python signer and tamper tests	Partial / superseded	Signing moved into backend provenance registration; a standalone image-embedding CLI is absent.
T3.2	FastAPI orchestrator with parallel dispatch	Complete / superseded	NestJS verification orchestrates local evidence and AI calls concurrently; FastAPI serves inference.
T1.2	CNN architecture and baseline training	Complete	ForensicFusion ConvNeXt/residual architecture and training pipeline are present.
T1.3	Full model training and fine-tuning	Complete with limitation	Checkpoint epoch 9 reports 94.84% validation accuracy and 98.84% PR-AUC; generalization is not established.
T3.3	Crypto verifier and registry integration	Complete / superseded	Manifest hashes, Ed25519 signatures, key history, ledger entries, and Merkle proofs are recomputed.
T2.3	React SPA full component development	Complete / superseded	Next.js dashboard implements user and administrator workflows across 27 page routes.
T2.4	Conditional rendering for three verdicts	Partial	Rich badges and reports exist; the original image blur/block interaction behavior is not implemented literally.
T1.4	ONNX export and inference integration	Partial / superseded	PyTorch FastAPI inference is integrated; ONNX export/runtime target is absent.
T3.4	Trust aggregation and verdict response	Complete	Configurable provenance/metadata and AI scoring returns explainable breakdowns, signals, classifications, and verdicts.

ID	Deliverable	Status	Current evidence
T1.5	Empirical evaluation S1-S6	Partial	Unit scenarios and targeted AI probes exist, but the complete standardized suite and signed evidence pack are absent.
T2.5	Frontend integration testing	Partial	Production build and lint pass; automated browser/E2E coverage across all adversarial scenarios is absent.
T3.5	Load tests, latency metrics, dissertation	Partial	Health, unit, CI, and operational checks exist; formal load/latency report and updated dissertation chapter remain.
T1.6	ArtiFact cross-generator fine-tuning	Partial	Checkpoint uses a broad training pipeline, but the observed Gemini false negative shows the cross-generator acceptance goal is unmet.
T1.7	GradCAM explainability	Not implemented	No heatmap generation, explain query parameter, or frontend visualization is present.
T2.6	Developer dashboard	Complete	API-key creation, one-time reveal, masking, rotation, revocation, scopes, limits, and usage views are implemented.
T2.7	PDF forensic report	Not implemented	Verification JSON can be downloaded; formatted PDF export is absent.
T3.6	API gateway middleware	Complete	Hashed and peppered keys, scopes, rate limiting, usage tracking, revocation, response headers, and Swagger are implemented.
T3.7	Docker deployment	Complete	Frontend, backend, AI service, Caddy proxy, health checks, limits, and environment templates are defined.

Deliverable summary

Status	Count	Weighted contribution
Complete	12	12.0
Partial / superseded	7	3.5
Not implemented	2	0.0
Total	21	15.5 / 21 = 73.8%

4. Functional Requirement Traceability

Use case	Requirement	Status	Traceability conclusion
UC-01	Sign image at origin	Partial / superseded	Backend issues signed manifests and ledger receipts, but does not return an image with embedded EXIF/C2PA signature.
UC-02	Upload image for verification	Complete	Dashboard validates and uploads images, shows progress/errors, and renders saved trust reports.
UC-03	Parallel verification pipeline	Complete / superseded	NestJS runs hashes/metadata and the HTTP AI request concurrently; uploads remain in memory.
UC-04	Cryptographic verification	Complete / superseded	Canonical manifest hash, Ed25519 signature, ledger link, Merkle proof, block hash, difficulty, and block signature are recomputed.
UC-05	AI anomaly detection	Partial	Production FastAPI inference exists, but architecture/runtime differ and cross-generator false negatives remain.
UC-06	Trust aggregation	Complete	Configurable 60/40 scoring, fallback weight redistribution, hard caps, signals, and four verdict levels are implemented.
UC-07	Conditional frontend rendering	Partial	Verdict/classification badges and detailed evidence views exist; original blocked-image blur behavior is absent.
UC-08	Manage API keys	Complete	Secure creation, one-time reveal, hashing, scopes, rotation, revocation, per-key limits, and usage views exist.
UC-09	External verification API	Complete	Scoped API-key endpoints expose verification and report retrieval with rate-limit metadata.
UC-10	History and analytics	Partial	Saved history, filtering, user/admin dashboards, and JSON export exist; PDF report export is absent.
UC-11	GradCAM heatmap	Not implemented	No GradCAM computation or response field is implemented.

5. Implemented System and Data Flows

5.1 Identity and access

1. A user signs up; the backend validates the request, creates or resolves the user role, stores a pending account, and sends a one-time verification code.
2. OTP validation activates the account. Login creates a server-recorded refresh session and issues HTTP-only authentication cookies.
3. Refresh rotates session credentials; logout and password changes revoke sessions. Administrator routes require the super-admin role.

5.2 Provenance registration and lineage

1. The caller uploads content or supplies a privacy-preserving content hash with declared source information.
2. The backend computes SHA-256 and, when bytes are present, pHash/dHash and normalized metadata. It derives a C2PA-aligned origin and action vocabulary.
3. A canonical manifest payload is hashed and signed with the platform Ed25519 key. The asset and versioned manifest are committed in a MongoDB transaction.
4. A ledger entry is appended with previous-entry linkage. Edited versions reference the current parent and preserve complete lineage.
5. Revocation creates a separately signed, ledger-anchored event without deleting the original historical record.

5.3 Verification and metadata return

1. Uploaded bytes are validated and processed in memory; ZTDPP does not persist verification image content.
2. SHA-256, pHash, dHash, EXIF, XMP, IPTC, PNG text, C2PA-container presence, and AI-generator hints are extracted while the backend calls the AI service.
3. The system searches for an exact claim and then an indexed pHash near match. Ambiguous claims require explicit selection.
4. Manifest and ledger evidence are recomputed rather than trusted from storage.
5. The trust engine returns metadata and AI scores, effective weights, verdict, classification, evidence signals, matched provenance, ledger/signature checks, input metadata, and processing time.
6. The complete policy and manifest snapshot is stored with the verification so historical reports remain explainable after settings change.

5.4 Storage map

Data category	Storage	Retention / handling
Accounts, roles, OTPs, sessions	MongoDB	Account and security lifecycle records; secrets are hashed or encrypted according to function.
API keys and usage	MongoDB	Only peppered hashes persist; plaintext is returned once. Usage is aggregated by key and date.
Assets and manifests	MongoDB	Versioned provenance, hashes, normalized metadata snapshots, signatures, status, and ownership.
Ledger entries and blocks	MongoDB	Append-oriented receipts, Merkle proofs, key IDs, block signatures, and optional future anchor.
Verification reports	MongoDB	Immutable evidence snapshot, scores, policy, signals, matched manifest, and requester attribution.
Uploaded verification bytes	Memory only	Processed during the request and discarded; no normal image persistence.
AI checkpoint	AI-Service model volume	Fixed server-controlled `best.pt`; excluded from Git and mounted read-only in Docker.
Optional administrator media	S3 utility	Separate administrative upload capability; not part of provenance verification retention.

6. Quality, Security, and Production Evidence

6.1 Verification evidence executed

Check	Result	Evidence
Backend automated tests	Passed	52 passed, 10 skipped, 6 suites passed, 1 suite skipped on 20 September 2026.
Backend lint	Passed	ESLint completed without reported errors during the implementation review.
Backend typecheck	Passed	TypeScript `tsc --noEmit` completed successfully.
Frontend lint	Passed	Next.js lint completed without warnings or errors.
Frontend production build	Passed	Next.js compiled, checked, and generated 24 static pages plus dynamic routes.
AI container build and health	Passed	Docker image built; `/health` confirmed checkpoint version, CPU device, and threshold 0.690918.
AI checkpoint integrity	Passed	Copied model SHA-256 matched the source checkpoint.
Formal load/latency suite	Pending	No retained benchmark demonstrating every SRS performance target under defined load.
Automated browser E2E	Pending	No full S1-S7 browser/API acceptance suite in the reviewed repositories.

6.2 AI model status

Metric	Checkpoint value
Validation accuracy	94.84%
Precision	95.67%
Recall	93.93%
F1	94.79%
ROC-AUC	98.74%
PR-AUC	98.84%
Decision threshold	0.690918

Model risk: An edited, watermark-removed Gemini image produced only 0.42% AI-generated probability. Three comparison photographs produced 0.51%, 0.86%, and 1.58%, so lowering or reversing the threshold would not safely correct the failure. The platform now labels a low output as 'No AI pixel signal', treats explicit generator filenames as untrusted evidence, and flags evidence/model disagreement. The checkpoint still requires retraining and generator-held-out evaluation.

6.3 Security posture

- Implemented strengths: HTTP-only session cookies, refresh-session rotation, password hashing, OTP expiry/consumption, API-key hashing with pepper, scoped authorization, throttling, CORS allowlists, request IDs, redacted logs, upload validation, circuit breaking, signing-key history, transaction use, and container hardening.
- Critical open issue: `POST /roles` is currently unauthenticated in `role.controller.ts`. Normal role creation should be limited to seed/migration logic or protected by super-admin authorization.
- Production requires durable Ed25519 key material and historical public-key retention. Ephemeral development keys cannot be used where old signatures must remain verifiable.
- Production MongoDB must support transactions and authenticated TLS connectivity. Database readiness must fail closed if transactional guarantees are unavailable.
- Horizontal scaling requires a single ledger sealer/leader and a shared rate-limit store such as Redis; the current in-process contracts are not sufficient for arbitrary multi-instance deployment.
- External C2PA containers are detected but their external trust chain is not validated. Hardware camera attestation is not implemented and must not be claimed.

7. Gap and Risk Register

ID	Priority	Risk	Impact	Required treatment
R1	Critical	Unauthenticated role creation	Privilege-control boundary can be altered through a public mutation route.	Remove the route from public modules or protect it with JWT plus super-admin guard; add regression test.
R2	High	AI cross-generator false negatives	New or post-processed generator output may receive a misleading low AI probability.	Retrain with modern generators and transformations; enforce generator-held-out promotion tests.
R3	High	SRS/SDS architecture drift	Evaluation, maintenance, and security review may rely on obsolete protocol descriptions.	Issue SRS/SDS v3.0 with formal change log and supervisor sign-off.
R4	High	GradCAM absent	UC-11, NFR-P8, and explainable forensic evidence are unmet.	Implement for the deployed ConvNeXt model or formally de-scope and replace with another explainability method.

ID	Priority	Risk	Impact	Required treatment
R5	High	Performance targets unverified	The system may fail latency or concurrency expectations despite functional correctness.	Run reproducible API, database, ledger, and inference benchmarks with retained reports.
R6	Medium	PDF report absent	T2.7 and the downloadable formal forensic report are incomplete.	Generate a signed or checksum-bound PDF from immutable verification snapshots.
R7	Medium	External C2PA validation absent	Container presence may be mistaken for verified third-party provenance.	Integrate a conforming C2PA verifier and a trust-store policy before claiming interoperability.
R8	Medium	Distributed operations not proven	Multiple API instances could race ledger sealing or apply inconsistent rate limits.	Use leader election/single sealer and Redis-backed limiting before horizontal scale.
R9	Medium	Full E2E suite absent	Unit success may not detect failures across browser, API, AI, database, and ledger boundaries.	Automate S1-S7 plus auth, revocation, recovery, ambiguity, and outage scenarios.

8. Completion Roadmap

Phase A - Production blockers

1. Protect or remove public role creation; audit all mutation routes for authentication and authorization.
2. Create production-only secrets, rotate any development credentials, provision durable ledger keys, and validate key backup/recovery.
3. Deploy an authenticated TLS MongoDB replica set and execute production migration/readiness checks.
4. Complete S1-S7 acceptance tests and add browser/API end-to-end automation for the critical user and administrator paths.
5. Run load and latency benchmarks against stated hardware and publish p50/p95/p99 results, error rate, throughput, CPU, and memory.

Phase B - AI quality and explainability

1. Build a versioned evaluation corpus containing Gemini/Imagen, DALL-E, Midjourney, Flux, Firefly, Stable Diffusion, camera images, and post-processing variants.
2. Use generator-held-out splits, calibrate the decision threshold on validation data, and run the untouched test set once per candidate release.
3. Promote a checkpoint only when per-generator recall and camera false-positive gates pass, including the known Gemini regression sample.
4. Implement GradCAM for the deployed ConvNeXt backbone, or obtain formal approval for a replacement explanation method and update UC-11/NFR-P8.

Phase C - Deliverables and governance

1. Generate a formal PDF forensic report from the immutable verification snapshot, including report ID, evidence, policy version, hashes, signatures, and ledger proof references.
2. Publish SRS v3.0 and SDS v2.0 reflecting the actual architecture, data model, API routes, trust policy, security boundaries, and deployment topology.
3. Update the dissertation with implementation evidence, architecture diagrams, model card, performance results, confusion matrices, adversarial outcomes, and limitations.
4. Conduct a final traceability review and obtain supervisor sign-off against the revised baseline.

9. Proposed Acceptance Gates

Gate	Acceptance evidence	Exit criterion
G1 Functional	Automated S1-S7 plus auth, keys, provenance edits, revocation, ledger, ambiguity, and outage cases	All critical paths pass with retained reports

Gate	Acceptance evidence	Exit criterion
G2 Security	Route authorization audit, secret scan, dependency audit, container review, negative upload/API tests	No unresolved critical/high exploitable finding
G3 AI quality	Generator-held-out confusion matrices, per-source recall, camera FPR, calibration, regression pack	Supervisor-approved metrics; known regression passes
G4 Performance	Reproducible p50/p95/p99 and throughput report on declared hardware	Every retained target passes or is formally revised
G5 Reliability	Database outage, AI timeout, signer-key absence, restart, and ledger recovery tests	Graceful degradation; no corrupt or partial records
G6 Documentation	Approved SRS/SDS revisions, deployment runbook, API guide, model card, dissertation chapter	Documents match deployed behavior and are signed off
G7 Deployment	Staging deployment through Caddy with TLS, backups, health probes, monitoring, and rollback	Repeatable deployment and rollback demonstrated

10. Final Assessment

ZTDPP is no longer an early prototype. The current repositories contain a coherent, security-aware provenance platform with substantially more capability than the FYP-I implementation described in the original reports. The strongest areas are provenance lifecycle management, ledger evidence, API-key integration, persisted trust reporting, and breadth of the user/admin interface.

The remaining work is concentrated rather than broad: reconcile the formal baseline with the actual design, close the public role endpoint, prove operational performance, complete the adversarial/E2E evidence pack, improve AI generalization, and decide the fate of GradCAM and PDF reporting. These items should be treated as acceptance gates rather than optional polish.

Recommended status: Feature-complete for an advanced academic demonstration after targeted blocker closure; not yet approved for unrestricted public production use.

Appendix A - Source and Evidence Register

Source	Version / snapshot	Use in this report
Software Requirements Specification	ZTDPP_SRS_v2.0.pdf, 10 May 2026	Functional use cases, NFRs, adversarial scenarios, deliverables, and schedule baseline.
Software Design Specification	ZTDPP_SDS_v1.0.pdf	Original component, sequence, interface, database, and deployment design.

Source	Version / snapshot	Use in this report
FYP-I Progress Report	ZTDPP_FYP1_Report (1).pdf, Spring 2026	Earlier milestone claims, baseline test results, architecture rationale, and training history.
Frontend repository	Local `main` at commit 6f3b816	Implemented routes, components, integration guides, and build evidence.
Backend repository	Local `main` at commit 927fd63	Domain modules, security controls, provenance, ledger, verification, AI client, deployment, and tests.
System architecture guide	docs/SYSTEM_ARCHITECTURE_AND_FLOWS.md	Current end-to-end flows, storage map, response shape, routes, and limitations.
Production readiness guide	docs/PRODUCTION_READINES S.md	Deployment controls, open production risks, and operational requirements.
AI model card	AI-Service/MODEL_CARD.md	Checkpoint metrics, known Gemini false negative, interpretation, and retraining gate.
Executed checks	20 September 2026	52 backend tests passed; lint/typecheck/build and AI container checks from the implementation review.

Appendix B - Status Definitions

Status	Definition
Complete	The deliverable or capability is implemented and supported by direct code, runtime, build, or test evidence.
Complete / superseded	The intended capability is implemented through a formally different architecture; requirements documents must be updated.
Partial	A meaningful implementation exists, but a required behavior, evidence package, or acceptance criterion remains.
Not implemented	The deliverable required by the baseline is absent from the reviewed implementation.